Final Report: CS 8803 DRL

Dennis Anthony, Aishani Chakraborty, Samuel Taubman
Georgia Institute of Technology

Abstract: Via reward shaping and experimenting with neural network architecture, we trained a model which results in faster convergence and better performance than the baseline. Our agent wins 79.5% of episodes against the baseline despite training for a similar amount of time and timesteps.

1 Overview

We made two key improvements to the baseline which resulted in a model which converges faster and performs better. First, we added dense rewards on top of the sparse goal reward. Second, we cut the size of the neural network from two hidden layers of size 256 to one of size 128. Both of these changes significantly improved both the stability of training and the speed of convergence. Combining them resulted in an agent with clearly improved performance over the baseline.

2 Method Description

Our method for training an agent that sufficiently beat the baseline involved training multiple models concurrently with varying approaches. After initial runs of the example scripts (DQN and PPO), we determined that we would proceed to experiment with PPO[1] due to faster convergence and stability. PPO (Proximal Policy Optimization) uses a clipped surrogate to multiply probability ratios between the old and new policies at each update, preventing large gradient updates and encouraging stable policy changes. We initially hypothesized that reward injection and larger models would perform best against the baseline.
Our rationale for implementing reward injection was to improve the learning signal by making rewards less sparse (i.e. going beyond only rewards for goals scored). We each tried a few different approaches, including player proximity to the ball, ball position, kick angle, and kick velocity. We each applied subsets of these injected rewards to both the team and individual player approaches. We also tried different neural network architectures, varying the number of hidden layers and the sizes of each.

2.1 Submitted Agent

Our final agent, named `Bert Agent`, was trained via self play with PPO[1] via the Ray Tune [2] library. Four distinct policies were trained simultaneously and competed against each other, but only the first was used in our final agent. The specific hyperparameters we used are found in the following table (default parameters found in ray tune source PPO source code).

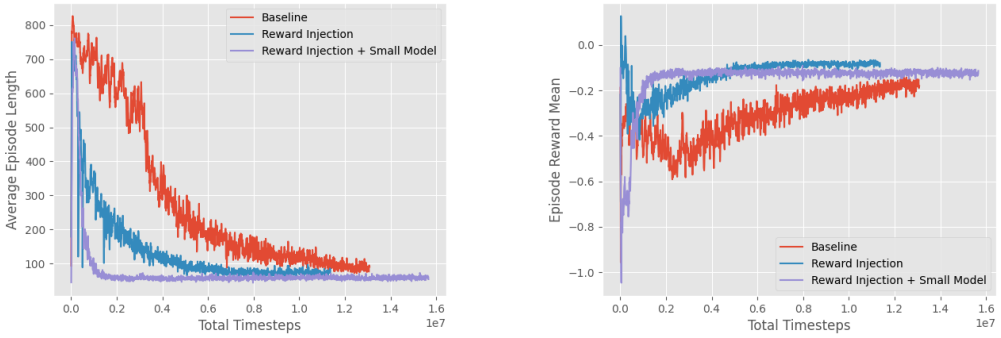
Parameter	Value	Notes
Learning Rate	5e-5	*Default*
Batch Size	4000	*Default*
Model Hidden Layers	[128]	Single hidden layer
Activation Function	Relu	Fully connected network
Total Timesteps	15 million	24 hours on PACE

`Bert Agent` was trained with three dense rewards: `Ball Proximity`, `Ball Position`, and `Existence`. `Ball Proximity` is designed to encourage agents to approach the ball, giving a max reward of 0.00025 next to ball and -0.00025 at a distance of 20. `Ball Position` is intended to

encourage the agent to move the ball towards the goal, giving a max reward of 0.0005 at the target goal, -0.0005 at the agent's goal, and 0 in the middle. Finally `Existence` is a constant penalty of -0.00005, further encouraging the model ot score quickly to end the episode.

All the code for our implementation can be found on GitHub at https://github.com/VirtualMe64/soccer-twos-starter. Specifically, our final agent is found in the folder bert_agent, our reward shaping code is located in wrappers.py, and our training code is located in injected_ray_ma_players3.py.

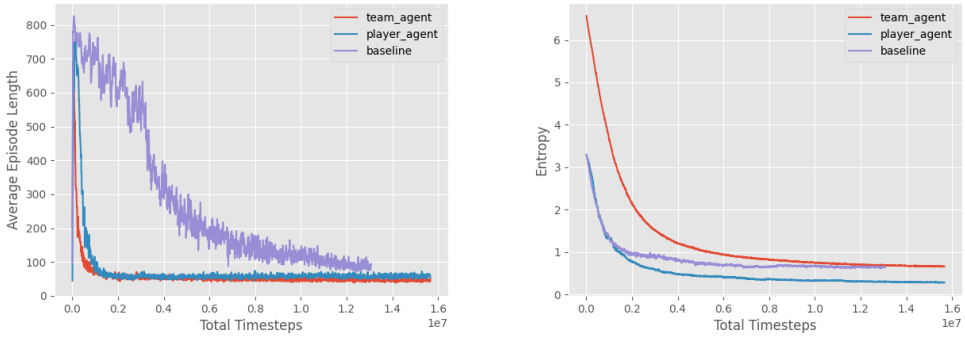
3 Preliminary Results



(a) Episode length vs. timesteps

(b) Mean reward* vs. timesteps

Figure 1: Small vs. large model performance metrics comparison.



(a) Episode length vs. timesteps

(b) Entropy vs. timesteps

Figure 2: Team agent vs. individual player performance metrics comparison.

3.1 Reward Injection Performance

As figure 1a shows, adding reward injection resulted in a much smoother training curve than baseline. It also reached shorter episode lengths (indicating agents are able to score quickly) much faster than the baseline. However, reward shaping alone only seemed to only make training faster and stabler, not result in a better agent. Per table 1, win rate was only 48.3% against the baseline.

We suspect this is because the dense rewards helped speed up the initial learning phase by encouraging the model to go near the ball and hit it to the right, but didn't provide much additional useful signal once the agent needed to learn how to score goals better. This is supported by figure 1a: there is an immediate sharp drop to 400 ts/episode that the baseline doesn't match until 4M timesteps of training later, but after the point the curves look relatively similar.

3.2 Neural Network Size

The baseline model has 2 hidden layers each with a layer size of 256. Our most important finding was that smaller network architectures (1 hidden layer, layer size of 128) significantly improved the speed of convergence and stability of the algorithm. We believe this is because over-parameterization can lead to slower learning and higher variance, ultimately "over-complicating" learning a relatively simple environment and leading to suboptimal policy updates. As shown in Figure 1b, we can see that reward injection using a smaller model converges much earlier around the millionth timestep, and its policy updates are less noisy in comparison to the larger-model approaches.

3.3 Team vs. Individual Training

Although we ultimately decided not to use it, we also experimented with training a team based policy, where a single model output the actions for both agents. Comparing results of the team and individual training approaches, we found that the team agent had a higher entropy throughout the training duration, as shown in Figure 2b. Although the baseline win rate of the player agent — which was the final agent we submitted — was higher, we believe that its lower entropy may have contributed to a failure mode we observed in tournament play where the agent repeatedly lost the kickoff due to a predictable initial move.

3.4 Empirical Performance

We evaluated our submitted agent which had both reward shaping and the smaller architecture (`Bert Agent`), an agent trained with only reward shaping (`Reward Shaping`), a team agent with reward shaping and smaller architecture (`Team Agent`), the provided baseline agent (`Baseline`), and an agent which took random actions (`Random`). For each pair, we simulated 1000 matches where the winner was the first to score a goal. The full results are in Table 1.

`Reward Shaping` alone resulted in an agent on par with but not exceeding the baseline (48.3% win rate), but `Bert Agent` clearly exceeded the performance of the baseline (79.5% win rate). Every agent was clearly better than random, with `Team Agent` performing the worst at a 93.4% win rate.

Table 1: Win Rates of Row Agent against Column Agent in 1000 Episodes where Winner is First to Score

Agent	Bert Agent	Team Agent	Reward Shaping	Baseline	Random
Bert Agent	—	**57.8**%	**77.7**%	**79.5**%	**98.4**%
Team Agent	**42.2**%	—	69.6%	70.8%	93.4%
Reward Shaping	22.3%	30.4%	—	48.3%	96.9%
Baseline	20.5%	29.2%	51.7%	—	97.3%
Random	1.6%	6.6%	3.1%	2.7%	—

4 Consent Statement

We consent to the instruction team using our Soccer-Twos submission (code, models, and report) for research and publication purposes. We understand this is voluntary, does not impact our course grades, and our models will be anonymized.

Team Name: Group 2 (Bert Agent)

Member 1: Samuel Taubman — staubman3@gatech.edu

Member 2: Aishani Chakraborty — achakraborty75@gatech.edu

Member 3: Dennis Anthony — danthony33@gatech.edu

Acknowledgments

This research was supported in part through research cyberinfrastructure resources and services provided by the Partnership for an Advanced Computing Environment (PACE) at the Georgia Institute of Technology, Atlanta, Georgia, USA. RRID:SCR_027619

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv.org*, Aug 2017. URL https://arxiv.org/abs/1707.06347.

[2] R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, and I. Stoica. Tune: A research platform for distributed model selection and training. *arXiv preprint arXiv:1807.05118*, 2018.